Khoulé Éducation

GUIDE DE L'ADMINISTRATEUR

Les requêtes SQL sous Supabase

Consulter les élèves, les intervenants et les cours, suivre les statistiques et modifier les données de la plateforme en toute sécurité.

34 requêtes prêtes à l'emploi

Version du 31/10/2026 · Projet Supabase « khoule-education »

Sommaire

1. Avant de commencer : à quoi sert l'éditeur SQL
2. Ouvrir l'éditeur et exécuter une requête
3. Les tables de la plateforme
4. Les bases du SQL en 5 exemples
5. Requêtes prêtes à l'emploi : utilisateurs
6. Requêtes prêtes à l'emploi : statistiques
7. Requêtes prêtes à l'emploi : cours et pédagogie
8. Modifier des données
9. Sécurité et bonnes pratiques
10. Erreurs fréquentes
11. Aide-mémoire des valeurs

> **Le site d'administration reste l'outil principal.** Valider un intervenant, affecter une classe, suspendre un compte ou publier une annonce se fait plus simplement (et avec notification automatique) depuis **khoule-education.vercel.app → espace Administrateur**. L'éditeur SQL sert pour les **listes complètes**, les **exports**, les **statistiques sur mesure** et les **corrections ponctuelles**.

1. Avant de commencer

Toutes les données de Khoulé Éducation (comptes, classes, cours, devoirs, notes, messages…) sont enregistrées dans une base de données **PostgreSQL** hébergée par Supabase. Le **SQL Editor** de Supabase permet d'interroger cette base directement avec le langage **SQL**.

Deux familles de requêtes

Type	Mots-clés	Effet
Lecture	SELECT	Affiche des données. **Aucun risque** : rien n'est modifié.
Modification	UPDATE, INSERT, DELETE	Change, ajoute ou supprime des données. **Effet immédiat et définitif** sur le site en ligne.

> ⚠ **Important : dans l'éditeur SQL, vous avez tous les droits.**
>
> Les règles de sécurité du site (un élève ne voit que ses données, etc.) **ne s'appliquent pas** dans l'éditeur : vous agissez comme propriétaire de la base. Une requête UPDATE ou DELETE sans **WHERE** modifie **toutes les lignes** de la table. Relisez toujours avant de cliquer sur Run.

Les données des élèves sont des données de mineurs : n'exportez et ne partagez que ce qui est strictement nécessaire à l'accompagnement scolaire.

2. Ouvrir l'éditeur et exécuter une requête

1	Connectez-vous sur **supabase.com** avec votre compte, puis ouvrez le projet **khoule-education**.
2	Dans le menu de gauche, cliquez sur **SQL Editor** (icône >_). Lien direct : https://supabase.com/dashboard/project/shgtnhypsernnsiwitqa/sql/new
3	Cliquez sur **+ New query**, puis collez ou tapez votre requête dans la zone de texte.
4	Cliquez sur **Run** (en bas à droite) ou appuyez sur **Ctrl + Entrée**.
5	Le résultat s'affiche sous la requête, sous forme de tableau.

Astuces utiles

Besoin	Comment faire
Exécuter une seule requête parmi plusieurs	Sélectionnez son texte à la souris, puis Ctrl + Entrée.
Garder une requête pour plus tard	Bouton **Save** : elle apparaît dans la liste « Private » à gauche.
Exporter le résultat (Excel)	Bouton **Export → Download CSV** au-dessus des résultats, puis ouvrir le fichier avec Excel ou Google Sheets.
Voir une table sans écrire de SQL	Menu **Table Editor** : affichage type tableur, filtres et tri.
Mettre un commentaire	Tout ce qui suit **--** sur une ligne est ignoré.
Message « This query has destructive operations »	Supabase vous demande de confirmer une modification : relisez, puis **Run this query**.

3. Les tables de la plateforme

Les tables de l'application se trouvent dans le schéma **public** : écrivez `public.nom_de_la_table`. Les e-mails sont dans `auth.users`.

Table	Contenu
`profiles`	Tous les comptes : prénom, nom, rôle (student / instructor / admin), statut
`auth.users`	Comptes de connexion : e-mail, date de création, dernière connexion
`students`	Fiche élève : classe, établissement, parent / tuteur, consentement
`instructors`	Fiche intervenant : type, études, spécialité, état de la candidature
`instructor_contacts`	Téléphone des intervenants (visible par l'administration seulement)
`instructor_availabilities`	Disponibilités des intervenants dans la semaine
`classes / subjects`	Classes (CI → Terminale) et matières
`class_subjects`	Programme : quelles matières dans quelle classe
`teaching_assignments`	Affectations : intervenant ↔ classe ↔ matière
`courses`	Cours : titre, classe, matière, intervenant, statut
`lessons / course_materials`	Leçons et supports d'un cours
`course_sessions / schedules`	Séances datées (salle, lien visio) et créneaux récurrents
`enrollments`	Inscriptions des élèves aux cours
`exercises / assignments`	Exercices et devoirs (avec date limite)
`submissions / corrections / grades`	Copies rendues, corrections et notes
`questions / answers`	Questions des élèves et réponses
`messages / notifications`	Messagerie et notifications
`announcements`	Annonces de l'administration
`library_documents`	Documents & annales déposés par les intervenants

Les tables sont reliées par des **identifiants** (colonnes id, profile_id, class_id, course_id…). Par exemple, un élève a une ligne dans `profiles` (son nom) et une ligne dans `students` (sa classe), avec le même identifiant.

4. Les bases du SQL en 5 exemples

Ces exemples suffisent à écrire la plupart des requêtes utiles. Copiez-les tels quels pour essayer.

Afficher une table entière (les 20 premières lignes)

● **Lecture seule (sans risque)**

SELECT choisit les colonnes (* = toutes), FROM la table, LIMIT le nombre de lignes.

```
select *
from public.classes
limit 20;
```

Choisir des colonnes et filtrer

● **Lecture seule (sans risque)**

WHERE garde seulement les lignes qui respectent la condition.

```
select first_name, last_name, status
from public.profiles
where role = 'student';
```

Trier et compter

● **Lecture seule (sans risque)**

ORDER BY trie le résultat ; count(*) compte les lignes.

```
select role, count(*) as nombre
from public.profiles
group by role
order by nombre desc;
```

Chercher un texte (sans tenir compte des majuscules)

● **Lecture seule (sans risque)**

ILIKE compare un texte ; % remplace n'importe quelle suite de caractères.

```
select first_name, last_name
from public.profiles
where last_name ilike '%cisse%';
```

Relier deux tables (JOIN)

● **Lecture seule (sans risque)**

Les élèves sont dans profiles (nom) et students (classe) : on les relie par l'identifiant.

```
select p.first_name, p.last_name, c.code as classe
from public.profiles p
join public.students s on s.profile_id = p.id
left join public.classes c on c.id = s.class_id
order by c.sort_order, p.last_name;
```

5. Requêtes prêtes à l'emploi : utilisateurs

Copiez la requête, adaptez la valeur entre apostrophes si besoin (code de classe, e-mail…), puis Run.

Tous les élèves avec leur classe et leur tuteur

● **Lecture seule (sans risque)**

Coordonnées complètes : à n'utiliser que pour l'accompagnement scolaire.

```
select p.first_name as prenom, p.last_name as nom, u.email,
       c.code as classe, s.school as etablissement,
       s.guardian_name as tuteur, s.guardian_phone as tel_tuteur,
       p.status as statut, p.created_at::date as inscrit_le
from public.profiles p
join auth.users u on u.id = p.id
join public.students s on s.profile_id = p.id
left join public.classes c on c.id = s.class_id
where p.role = 'student'
order by c.sort_order, p.last_name;
```

Les élèves d'une seule classe

● **Lecture seule (sans risque)**

Remplacez '3E' par le code voulu : CI, CP, CE1, CE2, CM1, CM2, 6E, 5E, 4E, 3E, 2ND, 1RE, TLE.

```
select p.first_name, p.last_name, u.email, s.guardian_phone
from public.profiles p
join auth.users u on u.id = p.id
join public.students s on s.profile_id = p.id
join public.classes c on c.id = s.class_id
where c.code = '3E' and p.status = 'active'
order by p.last_name;
```

Intervenants et état de leur candidature

● **Lecture seule (sans risque)**

pending = en attente, approved = validée, rejected = refusée.

```
select p.first_name as prenom, p.last_name as nom, u.email,
       ic.phone as telephone, i.instructor_type as type,
       i.specialty as specialite, i.application_status as candidature,
       p.status as statut, p.created_at::date as inscrit_le
from public.profiles p
join auth.users u on u.id = p.id
join public.instructors i on i.profile_id = p.id
left join public.instructor_contacts ic on ic.profile_id = p.id
where p.role = 'instructor'
order by i.application_status, p.created_at desc;
```

Candidatures en attente uniquement

● **Lecture seule (sans risque)**

```
select p.first_name, p.last_name, u.email, i.specialty, i.education_level
from public.profiles p
join auth.users u on u.id = p.id
join public.instructors i on i.profile_id = p.id
where i.application_status = 'pending'
order by p.created_at;
```

Trouver un compte à partir de son e-mail

● **Lecture seule (sans risque)**

```
select u.email, p.first_name, p.last_name, p.role, p.status,
       u.created_at::date as cree_le, u.last_sign_in_at as derniere_connexion
from auth.users u
left join public.profiles p on p.id = u.id
where u.email ilike '%gmail.com%';
```

Comptes suspendus

● **Lecture seule (sans risque)**

```
select p.first_name, p.last_name, p.role, u.email
from public.profiles p
join auth.users u on u.id = p.id
where p.status = 'suspended';
```

Liste des administrateurs

● **Lecture seule (sans risque)**

```
select p.first_name, p.last_name, u.email
from public.profiles p
join auth.users u on u.id = p.id
where p.role = 'admin';
```

6. Requêtes prêtes à l'emploi : statistiques

Pratique pour les bilans : le résultat s'exporte en CSV pour un rapport ou une réunion.

Nombre d'élèves par classe

● **Lecture seule (sans risque)**

```
select c.code as classe, count(s.profile_id) as eleves
from public.classes c
left join public.students s on s.class_id = c.id
group by c.code, c.sort_order
order by c.sort_order;
```

Résumé de la plateforme en une requête

● **Lecture seule (sans risque)**

```
select
  (select count(*) from public.profiles where role = 'student' and status = 'active') as eleves,
  (select count(*) from public.instructors where application_status = 'approved') as intervenants,
  (select count(*) from public.courses where status = 'published') as cours_publies,
  (select count(*) from public.enrollments where status = 'active') as inscriptions,
  (select count(*) from public.library_documents) as documents,
  (select count(*) from public.submissions where status = 'submitted') as copies_a_corriger;
```

Nombre d'inscrits par cours

● **Lecture seule (sans risque)**

```
select co.title as cours, c.code as classe, sj.name as matiere,
       count(e.id) filter (where e.status = 'active') as inscrits
from public.courses co
join public.classes c on c.id = co.class_id
join public.subjects sj on sj.id = co.subject_id
left join public.enrollments e on e.course_id = co.id
group by co.id, co.title, c.code, sj.name
order by inscrits desc;
```

Nouvelles inscriptions des 30 derniers jours

● **Lecture seule (sans risque)**

```
select created_at::date as jour, count(*) as nouveaux_comptes
from public.profiles
where created_at > now() - interval '30 days'
group by jour
order by jour;
```

Moyenne des notes par cours (sur 20)

● **Lecture seule (sans risque)**

```
select co.title as cours,
       round(avg(g.score / g.max_score * 20), 1) as moyenne_sur_20,
       count(g.id) as nombre_de_notes
from public.grades g
join public.courses co on co.id = g.course_id
group by co.title
order by moyenne_sur_20 desc;
```

7. Requêtes prêtes à l'emploi : cours et pédagogie

Tous les cours avec leur intervenant

● Lecture seule (sans risque)

```
select co.title as cours, c.code as classe, sj.name as matiere,
       p.first_name || ' ' || p.last_name as intervenant,
       co.status as statut, co.created_at::date as cree_le
from public.courses co
join public.classes c on c.id = co.class_id
join public.subjects sj on sj.id = co.subject_id
join public.profiles p on p.id = co.instructor_id
order by c.sort_order, sj.name;
```

Séances à venir (heure de Dakar)

● Lecture seule (sans risque)

```
select to_char(s.starts_at at time zone 'Africa/Dakar', 'DD/MM HH24:MI') as debut,
       co.title as cours, s.title as seance, s.room as salle,
       s.live_url as lien_visio
from public.course_sessions s
join public.courses co on co.id = s.course_id
where s.ends_at > now() and s.status = 'scheduled'
order by s.starts_at;
```

Devoirs rendus pas encore corrigés

● Lecture seule (sans risque)

```
select p.first_name || ' ' || p.last_name as eleve,
       e.title as devoir, co.title as cours,
       sub.submitted_at::date as rendu_le
from public.submissions sub
join public.assignments a on a.id = sub.assignment_id
join public.exercises e on e.id = a.exercise_id
join public.courses co on co.id = a.course_id
join public.profiles p on p.id = sub.student_id
where sub.status = 'submitted'
order by sub.submitted_at;
```

Toutes les notes d'un élève

● Lecture seule (sans risque)

Remplacez le nom de famille entre les % par celui de l'élève.

```
select e.title as devoir, co.title as cours,
       g.score || '/' || g.max_score as note, g.created_at::date as le
from public.grades g
join public.assignments a on a.id = g.assignment_id
join public.exercises e on e.id = a.exercise_id
join public.courses co on co.id = g.course_id
join public.profiles p on p.id = g.student_id
where p.last_name ilike '%ba%'
order by g.created_at desc;
```

Documents & annales déposés

● **Lecture seule (sans risque)**

```
select d.title as document, d.kind as type, d.year as annee,
       c.code as classe, sj.name as matiere,
       p.first_name || ' ' || p.last_name as depose_par, d.created_at::date as le
from public.library_documents d
join public.classes c on c.id = d.class_id
join public.subjects sj on sj.id = d.subject_id
left join public.profiles p on p.id = d.uploaded_by
order by d.created_at desc;
```

Affectations des intervenants

● **Lecture seule (sans risque)**

```
select p.first_name || ' ' || p.last_name as intervenant,
       c.code as classe, sj.name as matiere
from public.teaching_assignments ta
join public.profiles p on p.id = ta.instructor_id
join public.classes c on c.id = ta.class_id
join public.subjects sj on sj.id = ta.subject_id
order by c.sort_order, sj.name;
```

Disponibilités des intervenants dans la semaine

● **Lecture seule (sans risque)**

day_of_week : 0 = dimanche, 1 = lundi ... 6 = samedi.

```
select p.first_name || ' ' || p.last_name as intervenant,
       case a.day_of_week when 0 then 'Dimanche' when 1 then 'Lundi'
         when 2 then 'Mardi' when 3 then 'Mercredi' when 4 then 'Jeudi'
         when 5 then 'Vendredi' else 'Samedi' end as jour,
       to_char(a.start_time, 'HH24:MI') || ' - ' || to_char(a.end_time, 'HH24:MI') as horaire,
       a.mode as format
from public.instructor_availabilities a
join public.profiles p on p.id = a.instructor_id
order by p.last_name, (a.day_of_week + 6) % 7, a.start_time;
```

8. Modifier des données

Avant toute modification :

1) Lancez d'abord un **SELECT** avec le même WHERE pour voir les lignes concernées.
2) Remplacez les exemples (`intervenant@exemple.com`, titres…) par les vraies valeurs.
3) En cas de doute, utilisez une **transaction** (section 9) pour tester sans rien enregistrer.
4) Préférez le site quand c'est possible : il envoie aussi les **notifications** aux personnes concernées.

Valider la candidature d'un intervenant

● **Modifie les données**

Deux requêtes à exécuter ensemble. Remplacez l'adresse e-mail. Pensez ensuite à l'affecter (requête suivante ou page Affectations du site).

```
update public.instructors
   set application_status = 'approved', reviewed_at = now()
 where profile_id = (select id from auth.users where email = 'intervenant@exemple.com');

update public.profiles set status = 'active'
 where id = (select id from auth.users where email = 'intervenant@exemple.com');
```

Affecter un intervenant à une classe et une matière

● **Modifie les données**

```
insert into public.teaching_assignments (instructor_id, class_id, subject_id)
select u.id, c.id, s.id
from auth.users u, public.classes c, public.subjects s
where u.email = 'intervenant@exemple.com'
  and c.code = 'TLE' and s.name = 'Mathématiques';
```

Refuser une candidature

● **Modifie les données**

```
update public.instructors
   set application_status = 'rejected',
       rejection_reason = 'Profil incomplet', reviewed_at = now()
 where profile_id = (select id from auth.users where email = 'intervenant@exemple.com');
```

Suspendre un compte (ou le réactiver)

● **Modifie les données**

Mettez 'active' à la place de 'suspended' pour réactiver.

```
update public.profiles set status = 'suspended'
 where id = (select id from auth.users where email = 'utilisateur@exemple.com');
```

Changer la classe d'un élève (passage en classe supérieure)

● **Modifie les données**

```
update public.students
   set class_id = (select id from public.classes where code = '2ND')
 where profile_id = (select id from auth.users where email = 'eleve@exemple.com');
```

Publier ou archiver un cours

● **Modifie les données**

Valeurs possibles : 'draft' (brouillon), 'published' (publié), 'archived' (archivé).

```
update public.courses set status = 'archived'
 where title = 'Titre exact du cours';
```

Publier une annonce pour tout le monde

● **Modifie les données**

audience : 'all', 'students', 'instructors' ou 'class' (avec class_id).

```
insert into public.announcements (title, body, audience, pinned)
values ('Reprise des cours', 'Les cours reprennent lundi à 17 h.', 'all', true);
```

Donner les droits d'administrateur

● **Modifie les données**

À n'utiliser que pour une personne de confiance : elle aura accès à tout.

```
update public.profiles set role = 'admin', status = 'active'
 where id = (select id from auth.users where email = 'personne@exemple.com');
```

9. Sécurité et bonnes pratiques

Tester une modification sans risque (transaction)

● Lecture seule (sans risque)

BEGIN ouvre une transaction ; ROLLBACK annule tout. Quand le résultat est bon, remplacez ROLLBACK par COMMIT pour enregistrer.

```
begin;

update public.courses set status = 'archived'
 where title = 'Titre exact du cours';

select title, status from public.courses where title = 'Titre exact du cours';

rollback;   -- remplacez par commit; pour enregistrer
```

Compter avant de modifier ou de supprimer

● Lecture seule (sans risque)

Lancez d'abord le SELECT avec le même WHERE : si le nombre n'est pas celui attendu, n'allez pas plus loin.

```
select count(*) from public.enrollments
where status = 'dropped';
```

Les règles d'or

✔	Toujours un **WHERE** dans un UPDATE ou un DELETE.
✔	Un **SELECT** avec le même WHERE avant de modifier, pour vérifier le nombre de lignes.
✔	Exporter (CSV) les données importantes avant une grosse modification.
✘	Ne jamais modifier ni supprimer directement dans `auth.users` : utilisez Authentication → Users, ou le bouton de suppression du site (qui supprime aussi les fichiers).
✘	Ne jamais désactiver la sécurité d'une table (« Disable RLS ») : les données des élèves deviendraient lisibles par tous.
✘	Ne jamais copier la clé **service_role** (Project Settings → API) dans un message, un fichier partagé ou le site.
✘	Ne pas modifier la structure des tables (ALTER, DROP) : l'application en dépend.

Sauvegardes : le plan gratuit de Supabase ne permet pas de revenir en arrière après une erreur. Pour les données importantes, gardez des exports CSV réguliers (requêtes des sections 5 à 7).

10. Erreurs fréquentes

Si une requête échoue, Supabase affiche un message en anglais en bas de l'éditeur. Voici les plus courants et comment les corriger.

Message affiché	Signification et solution
`syntax error at or near …`	Faute de frappe : virgule ou guillemet oublié, mot mal écrit. Les textes s'écrivent entre apostrophes droites : 'texte'.
`relation "…" does not exist`	Nom de table incorrect. Préfixez par public. (ex. public.courses) ou auth. pour auth.users.
`column "…" does not exist`	Nom de colonne incorrect : consultez la table dans le Table Editor.
`violates check constraint`	Valeur non autorisée (ex. un statut qui n'existe pas). Utilisez exactement les valeurs indiquées dans ce guide.
`violates foreign key constraint`	Vous faites référence à un élément qui n'existe pas (ex. une classe ou un utilisateur introuvable).
`duplicate key value violates unique constraint`	L'élément existe déjà (ex. affectation déjà faite).
`UPDATE 0 / Success. No rows returned`	La requête a fonctionné mais n'a trouvé aucune ligne : vérifiez l'e-mail ou le titre utilisé dans le WHERE.

11. Aide-mémoire des valeurs

Les colonnes ci-dessous n'acceptent que ces valeurs exactes (en minuscules, entre apostrophes).

Colonne	Valeurs possibles
`profiles.role`	'student' (élève) · 'instructor' (intervenant) · 'admin' (administrateur)
`profiles.status`	'active' (actif) · 'pending' (en attente) · 'suspended' (suspendu)
`instructors.application_status`	'pending' (en attente) · 'approved' (validée) · 'rejected' (refusée)
`instructors.instructor_type`	'enseignant' · 'professeur' · 'etudiant' · 'autre'
`courses.status`	'draft' (brouillon) · 'published' (publié) · 'archived' (archivé)
`course_sessions.status`	'scheduled' (programmée) · 'done' (terminée) · 'cancelled' (annulée)
`enrollments.status`	'active' (inscrit) · 'dropped' (a quitté le cours)
`submissions.status`	'submitted' (à corriger) · 'corrected' (corrigée)
`questions.status`	'open' (en attente) · 'answered' (répondue) · 'closed' (fermée)
`announcements.audience`	'all' · 'students' · 'instructors' · 'class' (avec class_id)
`library_documents.kind`	'annale' · 'corrige' · 'cours' · 'exercices' · 'fiche' · 'autre'
`*.day_of_week`	0 dimanche · 1 lundi · 2 mardi · 3 mercredi · 4 jeudi · 5 vendredi · 6 samedi
`classes.code`	CI · CP · CE1 · CE2 · CM1 · CM2 · 6E · 5E · 4E · 3E · 2ND · 1RE · TLE

Mémo des mots-clés

Mot-clé	Rôle	Exemple
SELECT … FROM	Lire des colonnes d'une table	select title from public.courses
WHERE	Filtrer les lignes	where status = 'published'
ORDER BY	Trier (desc = décroissant)	order by created_at desc
LIMIT	Limiter le nombre de lignes	limit 50
JOIN … ON	Relier deux tables	join public.classes c on c.id = s.class_id
COUNT / GROUP BY	Compter par groupe	select role, count(*) … group by role
ILIKE '%…%'	Rechercher un texte	where last_name ilike '%diop%'
UPDATE … SET	Modifier des lignes	update … set status = 'active' where …
INSERT INTO	Ajouter une ligne	insert into … (col) values ('…')
BEGIN / ROLLBACK	Tester puis annuler	begin; … rollback;